Práctica 1 de Big Data y Minería de datos: Conceptos básicos.

Autores: José Antonio Ruipérez Valiente, Mariano Albaladejo González y Manuel Jesús Gómez Moratilla

En las prácticas de la asignatura de Big Data y Minería de Datos trabajaremos con Python a través de Google Colab. En esta primera práctica haremos un repaso de algunos de los conceptos esenciales, pero es muy recomendable que volváis a echar un vistazo a las prácticas de otras asignaturas. Las prácticas de la asignatura Visualización de Datos son especialmente relevantes para poder entender las prácticas de esta asignatura.

1.Introducción a Google Colab

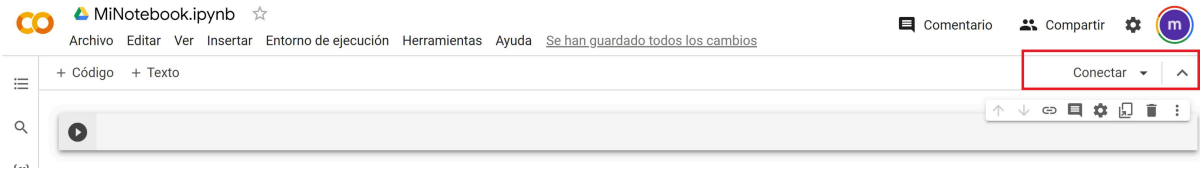
Google Colab, también conocido como Google Colaboratory, es un entorno de desarrollo integrado en la nube que permite escribir, ejecutar y colaborar en código Python. Es una herramienta proporcionada por Google de manera gratuita que se basa en el popular proyecto de código abierto Jupyter Notebook. La principal diferencia entre Jupyter Notebook y Google Colab se encuentra en dónde se ejecuta el código. Mientras que con un Jupyter Notebook el código se ejecuta en nuestra máquina y en Google Colab se ejecuta en los servidores de Google.

Algunas características clave de Google Colab son:

1. **Acceso en la nube.** Google Colab se ejecuta en servidores de Google, lo que significa que no es necesario instalar ni configurar nada en nuestros dispositivos. Solo necesitamos un navegador web para acceder a la plataforma.
2. **Recursos de hardware.** Google Colab proporciona acceso a recursos computacionales en la nube, lo que permite ejecutar código en máquinas potentes sin preocuparnos por la capacidad de nuestros dispositivos.
3. **Notebooks interactivos.** Al igual que Jupyter Notebook, Google Colab permite crear documentos interactivos que combinan código, texto explicativo y elementos visuales, como gráficos y diagramas.
4. **Integración con Google Drive.** Los notebooks de Google Colab se almacenan en nuestra cuenta de Google Drive, lo que facilita la colaboración y el acceso a los proyectos desde cualquier lugar.
5. **Bibliotecas preinstaladas.** Google Colab ya incluye muchas bibliotecas populares de Python preinstaladas, lo que facilita la importación y el uso de herramientas de análisis de datos y aprendizaje automático.
6. **Compatibilidad con GPU y TPU.** Google Colab proporciona acceso a unidades de procesamiento gráfico (GPU) y unidades de procesamiento tensorial (TPU) de forma gratuita, lo que es útil para acelerar cálculos intensivos, como entrenamiento de modelos de aprendizaje profundo.

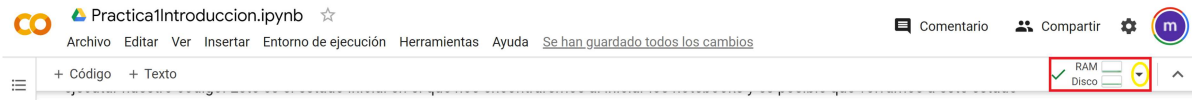
En los entornos de ejecución de Google Colab podemos encontrarnos en dos estados diferentes dependiendo de si tenemos o no una sesión activa:

1. **Sin sesión activa.** En este estado podemos visualizar el notebook y las salidas de las distintas celdas si estas se han ejecutado previamente. Sin embargo, en este estado no disponemos de un entorno de ejecución asignado. Por lo tanto, no tenemos una máquina en la nube en la que ejecutar nuestro código. Este es el estado inicial en el que nos encontraremos al iniciar los notebooks y es posible que volvamos a este estado por diferentes motivos, por ejemplo, que excedamos los recursos disponibles, que tengamos la sesión mucho tiempo abierta o se saturen los servidores de Google Colab. Podemos identificar este estado porque en la esquina superior derecha aparece el botón *Conectar*.



Para activar una sesión con un entorno de ejecución basta con que intentemos ejecutar una línea de código o pulsemos en el botón *Conectar* situado en la esquina superior derecha.

2. **Sesión activa**. Dentro de este estado tenemos un entorno de ejecución y unos recursos computacionales asignados. Por lo tanto, tenemos una máquina capaz de ejecutar nuestro código. Cada vez que se inicia una sesión todos los datos de carácter temporal desaparecen. Esto incluye variables, ficheros cargados y librerías adicionales instaladas. Cambiar de sesión es similar a migrar a un nuevo equipo por lo que no habrá memoria de lo ejecutado previamente. Es posible que perdamos las sesiones activas, por ejemplo, si cerramos la pestaña con el código en ejecución durante unos minutos.

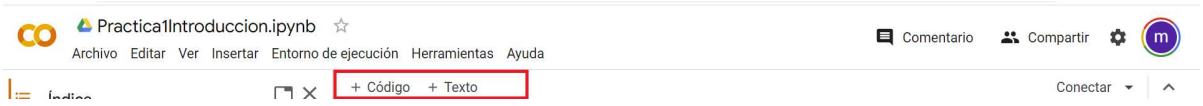


2.Funcionalidades de Google Colab y de los notebooks

Los notebooks combinan bloques (o celdas) que pueden ser principalmente de dos tipos:

- Bloques de **texto** en formato *Markdown*. Estos bloques también soportan imágenes. Es el caso de todas las celdas anteriores.
- Bloques de **código**, en nuestro caso en Python. La siguiente celda es un ejemplo de celda con código Python. Para ejecutar estas celdas y obtener su salida debemos tener una sesión activa. Sin una sesión activa sólo podremos visualizar un resultado si esta se ha ejecutado previamente.

En la barra superior podemos encontrar dos botones para añadir celdas de código o de texto:




```
In [1]: 1 print("Hola mundo!")
```

```
Hola mundo!
```

Existen diversas formas de ejecutar una celda:

- **Ctrl + Enter**: ejecuta la celda actual.
- **Shift + Enter**: ejecuta la celda actual y pasa a la siguiente celda (sin evaluarla). Si nos encontramos en la última celda, creará una nueva celda.
- **Alt + Enter**: ejecuta la celda actual e inserta una nueva celda a continuación.
- **Botón Ejecutar celda**: este botón se sitúa a la izquierda de cada celda y ejecuta dicha celda.
- **Menú Entorno de Ejecución**: en este menú tenemos diversas opciones, incluyendo ejecutar todas las celdas desde la actual o todas las anteriores.

Cada celda tiene una barra de herramientas donde podemos realizar las siguientes acciones:



1. Mover la celda actual hacia arriba.
2. Mover la celda actual hacia abajo.
3. Obtener un enlace directo (URL) a la celda actual.
4. Poner comentario en la celda actual.
5. Acceder a la configuración del editor, donde se podrá seleccionar la fuente y sangría.
6. Abrir una pestaña para la celda actual.
7. Eliminar la celda actual.
8. Acceder a opciones adicionales.

También tenemos una barra de herramientas situada en la parte superior izquierda con las siguientes funcionalidades:



1. Índice del notebook
2. Búsqueda dentro del notebook
3. Fragmentos de código. Biblioteca de ejemplos de código listos para usar
4. Secretos. Añadir código oculto almacenando variables de entorno, rutas de archivos o claves
5. Archivos

Finalmente, en la barra inferior tenemos el inspector de variables y la terminal. El inspector de variables nos permitirá revisar el valor de las diferentes variables de nuestra sesión activa, facilitándonos la depuración. La terminal nos permite ejecutar comandos de Linux, lo que nos habilita otra forma de instalar librerías, navegar por el sistema de archivos y monitorizar recursos.



3.Gestión de archivos en Google Colab

Podemos cargar los archivos que necesitemos (principalmente conjuntos de datos) de dos formas distintas:

1. Subiendo los archivos directamente a la pestaña *archivos* (accesible desde la barra de herramientas situada en la parte superior izquierda). Esta no es la opción recomendable para estas prácticas ya que cada vez que iniciéis una nueva sesión tendréis que subir de nuevo los archivos. En esta signatura trabajaremos con conjuntos de datos pesados que pueden requerir bastante tiempo para subirlos, por ello, no es la opción recomendada.
2. Subiendo los archivos a Google Drive y montándolo en la sesión actual. En esta opción sólo subiremos una vez el conjunto de datos, sin embargo, si perdemos la sesión tendremos que volver a montar el acceso a Google Drive. Para montar este acceso,

únicamente hay que ejecutar la siguiente celda:

In [2]:
```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

4.Repaso de Python

4.1. Estructuras de datos

Las estructuras de datos son herramientas que posibilitan el almacenamiento eficiente de información con una estructura definida. Existen diversos tipos de estas estructuras, cada una con propósitos específicos según la aplicación y los requisitos.

4.1.1. Listas

En Python, una lista es una estructura de datos que permite almacenar una colección ordenada de elementos. Estos elementos pueden ser de cualquier tipo, como números, cadenas de texto, booleanos e incluso otras listas u objetos más complejos. Los elementos de la lista pueden estar repetidos. Las listas en Python son flexibles y versátiles, lo que las hace muy útiles para manejar conjuntos de datos dinámicos y variables.

Las listas se definen utilizando corchetes [ ] y separando los elementos con comas.

In [3]:
```
# Creamos una lista
mi_lista = ["Elemento1", "ElementoRepetido", "Elemento2", "ElementoRepetid

# Mostramos toda la lista
print("La lista contiene: ", mi_lista)

# Mostramos el primer elemento
print("El primer elemento es: ", mi_lista[0])

```

```
La lista contiene:  ['Elemento1', 'ElementoRepetido', 'Elemento2', 'ElementoR
epetido']
El primer elemento es:  Elemento1
```

Ejercicio 1

1. Crea una lista con el nombre de 5 ciudades.
2. Muestra la ciudad de la última posición de la lista.
3. Elimina la ciudad situada en la segunda posición de la lista.
4. Añade una nueva ciudad a la lista.

4.1.2 Conjuntos

En Python, un conjunto es una estructura de datos que almacena una colección desordenada de elementos únicos. A diferencia de las listas, en las que el orden de los elementos es importante y puede haber duplicados, los conjuntos garantizan que cada elemento aparezca solo una vez y no tienen un orden específico.

Los conjuntos son útiles cuando necesitamos almacenar una colección de elementos sin preocuparnos por el orden y la duplicación. Esto puede ser útil para verificar rápidamente la existencia de un elemento en el conjunto o para realizar operaciones matemáticas como intersección, unión y diferencia entre conjuntos.

Los conjuntos en Python se definen utilizando llaves {}

In [4]:
```python
# Creamos un conjunto
mi_conjunto = {1, 2, 3, 4, 5, 8}
print("Mi conjunto contiene inicialmente: ", mi_conjunto)

# Agregamos elementos al conjunto
mi_conjunto.add(6)
mi_conjunto.add(3)

print("Mi conjunto contiene finalmente: ", mi_conjunto)

```

```
Mi conjunto contiene inicialmente:  {1, 2, 3, 4, 5, 8}
Mi conjunto contiene finalmente:  {1, 2, 3, 4, 5, 6, 8}
```

Ejercicio 2

1. Sobre el conjunto "mi conjunto" elimina el número 3.
2. Sobre el conjunto "mi conjunto" añade el número 1. ¿Observáis algo extraño?

4.1.3. Diccionarios

En Python, un diccionario es una estructura de datos que permite almacenar y organizar elementos en pares clave-valor. Cada elemento en un diccionario consta de una clave y un valor asociado. Las claves actúan como identificadores únicos para acceder a los valores correspondientes de manera eficiente.

Los diccionarios son especialmente útiles cuando necesitamos asociar información entre dos elementos de manera que podamos acceder rápidamente a los valores usando sus claves en lugar de índices numéricos.

Los diccionarios en Python se definen utilizando llaves {}. Cada par clave-valor se separa por dos puntos : y los pares se separan entre sí por comas.

A continuación, tienes un ejemplo de un diccionario donde las claves son el nombre de un equipo de fútbol y el valor es el número de puntos en la liga.

In [5]:
```
diccionario = {
    "FC Barcelona": 12,
    "Real Madrid CF": 8,
    "Atletico de Madrid": 8,
}

diccionario
```

Out[5]:
```
{'FC Barcelona': 12, 'Real Madrid CF': 8, 'Atletico de Madrid': 8}
```

Aquí tienes un diccionario más complejo, donde el campo valor del primer diccionario es otro diccionario.

In [6]:

```python
# Creamos un diccionario de información de coches
coches = {
    "coche1": {
        "marca": "Toyota",
        "modelo": "Corolla",
        "año": 2022,
        "color": "Azul"
    },
    "coche2": {
        "marca": "Ford",
        "modelo": "Mustang",
        "año": 2021,
        "color": "Rojo"
    },
    "coche3": {
        "marca": "BMW",
        "modelo": "Serie3",
        "año": 2023,
        "color": "Azul"
    }
}

# Acceder a valores de un coche específico mediante claves
print("Coche 1 - Marca:", coches["coche1"]["marca"])
print("Coche 2 - Modelo:", coches["coche2"]["modelo"])

# Agregar información adicional a un coche
coches["coche1"]["precio"] = 25000
coches["coche2"]["precio"] = 45000

# Mostrar la información actualizada de los coches
print("Información de todos los coches:", coches)


```

```
Coche 1 - Marca: Toyota
Coche 2 - Modelo: Mustang
Información de todos los coches: {'coche1': {'marca': 'Toyota', 'modelo': 'Co
rolla', 'año': 2022, 'color': 'Azul', 'precio': 25000}, 'coche2': {'marca':
'Ford', 'modelo': 'Mustang', 'año': 2021, 'color': 'Rojo', 'precio': 45000},
'coche3': {'marca': 'BMW', 'modelo': 'Serie3', 'año': 2023, 'color': 'Azul'}}
```

Ejercicio 3

1. Crea un diccionario con tres ejemplos donde la clave sea el nombre de una tienda y en su campo valor contenga los atributos el tamaño de la tienda, la ciudad donde se sitúa, el número de empleados y la facturación anual.
2. Elimina uno de los ejemplos del diccionario.

4.1.4 Arrays

Los arrays son una estructura fundamental en cualquier lenguaje de programación. En Python están incluidos mediante la librería NumPy. La principal ventaja de utilizar arrays frente a listas es que permite aplicar operaciones matriciales y son mucho más eficientes tanto en memoria como en cálculo frente a las listas. Sin embargo, todos los elementos del array deben ser del mismo tipo.

Tal y como están diseñados los arrays las operaciones de acceso y modificación de los elementos del array son muy rápidas, pero incrementar una posición o eliminar una posición del array es más costosa.

In [7]:
```
# Importamos la librería de NumPy
import numpy as np

# Creamos un array de NumPy a partir de una lista
mi_array = np.array([1, 2, 3, 4, 5])

# Realizamos operaciones en el array
mi_array_doble = mi_array * 2

# Mostramos el array resultante
print(mi_array_doble)

```

```
[ 2  4  6  8 10]
```

Ejercicio 4

1. Crea un array de números aleatorios con las dimensiones 4 x 3 e inicialízalo con los números que quieras.
2. Accede al elemento situado en la posición (2, 2) y sustitúyelo por un entero aleatorio entre 1 y 100.

4.1.5 DataFrames

Un DataFrame es una estructura de datos bidimensional que se encuentra en la librería Pandas. Un DataFrame se asemeja a una hoja de cálculo o a una tabla en una base de datos, y consta de filas y columnas etiquetadas que permiten almacenar, acceder y manipular datos de manera eficiente. Esta estructura de datos será muy utilizada durante las prácticas de esta asignatura.

In [8]:

```python
import pandas as pd

# Creamos un diccionario con los datos de alumnos
datos_alumnos = {
    "DNI": ["12345678A", "23456789B", "34567890C", "45678901D", "56789012E
            "67890123F", "78901234G", "89012345H", "90123456I", "01234567J
    "Nombre y apellidos": ["Juan Pérez", "Ana García", "Carlos Martínez",
                           "Pedro López", "Laura Fernández", "Marta Gonzá
    "Nota lengua": [7.5, 8.0, 6.5, 9.2, 5.7, 8.9, 7.2, 6.8, 9.5, 8.3],
    "Nota matematicas": [6.8, 7.1, 8.5, 6.3, 5.8, 4.2, 2.7, 8.0, 7.4, 6.9]
    "Nota ingles": [8.2, 7.9, 8.0, 9.0, 6.5, 7.1, 8.3, 8.5, 9.2, 7.8],
    "Nota fisica": [7.0, 6.3, 7.8, 6.9, 8.1, 7.7, 6.5, 7.2, 8.0, 6.8],
    "Nota quimica": [8.9, 7.4, 9.1, 8.6, 7.0, 8.4, 7.6, 9.0, 8.8, 7.5]
}

# Creamos el DataFrame a patir del diccionario
df_alumnos = pd.DataFrame(datos_alumnos)

# Mostramos el DataFrame
df_alumnos

```

Out[8]:

	DNI	Nombre y apellidos	Nota lengua	Nota matematicas	Nota ingles	Nota fisica	Nota quimica
0	12345678A	Juan Pérez	7.5	6.8	8.2	7.0	8.9
1	23456789B	Ana García	8.0	7.1	7.9	6.3	7.4
2	34567890C	Carlos Martínez	6.5	8.5	8.0	7.8	9.1
3	45678901D	Luisa Sánchez	9.2	6.3	9.0	6.9	8.6
4	56789012E	María Rodríguez	5.7	5.8	6.5	8.1	7.0
5	67890123F	Pedro López	8.9	4.2	7.1	7.7	8.4
6	78901234G	Laura Fernández	7.2	2.7	8.3	6.5	7.6
7	89012345H	Marta González	6.8	8.0	8.5	7.2	9.0
8	90123456I	David Ruiz	9.5	7.4	9.2	8.0	8.8
9	01234567J	Sofía Torres	8.3	6.9	7.8	6.8	7.5

En Python, existen tres métodos que se utilizan para acceder a los datos en un DataFrame, pero se utilizan de manera ligeramente diferente:

- **[ ]**: Se utiliza para seleccionar columnas de un DataFrame por nombre. También se puede utilizar para seleccionar filas condicionalmente.
- **loc**: Se utiliza para seleccionar filas y columnas por etiquetas (nombres de filas según el índice del DataFrame y nombres de columnas).
- **iloc**: Se utiliza para seleccionar filas y columnas por índices enteros (posiciones). Utiliza índices enteros para seleccionar datos, como si estuvieras trabajando con una matriz.

A continuación, vamos a ver un ejemplo de las tres formas de acceder a los elementos del DataFrame.

Pero antes de comenzar vamos a cambiar el índice al DataFrame para hacer el ejemplo más interesante.

In [9]:
```
# Establecemos el índice numérico explícito
df_alumnos.index = [1, 2, 3, 5, 8, 10, 12, 16, 17, 14]
df_alumnos
```

Out[9]:

	DNI	Nombre y apellidos	Nota lengua	Nota matematicas	Nota ingles	Nota fisica	Nota quimica
1	12345678A	Juan Pérez	7.5	6.8	8.2	7.0	8.9
2	23456789B	Ana García	8.0	7.1	7.9	6.3	7.4
3	34567890C	Carlos Martínez	6.5	8.5	8.0	7.8	9.1
5	45678901D	Luisa Sánchez	9.2	6.3	9.0	6.9	8.6
8	56789012E	María Rodríguez	5.7	5.8	6.5	8.1	7.0
10	67890123F	Pedro López	8.9	4.2	7.1	7.7	8.4
12	78901234G	Laura Fernández	7.2	2.7	8.3	6.5	7.6
16	89012345H	Marta González	6.8	8.0	8.5	7.2	9.0
17	90123456I	David Ruiz	9.5	7.4	9.2	8.0	8.8
14	01234567J	Sofía Torres	8.3	6.9	7.8	6.8	7.5

Acceso mediante [ ]

In [10]:
```
print("Acceso con []:")
df_alumnos['Nombre y apellidos']      # Accedemos a la columna "Nombre"

```

```
Acceso con []:
```

Out[10]:

	Nombre y apellidos
1	Juan Pérez
2	Ana García
3	Carlos Martínez
5	Luisa Sánchez
8	María Rodríguez
10	Pedro López
12	Laura Fernández
16	Marta González
17	David Ruiz
14	Sofía Torres

dtype: object

In [11]:
```
print("Acceso con []:")
df_alumnos[:5]          # Accedemos a las cinco primeras filas
```

```
Acceso con []:
```

Out[11]:

	DNI	Nombre y apellidos	Nota lengua	Nota matematicas	Nota ingles	Nota fisica	Nota quimica
1	12345678A	Juan Pérez	7.5	6.8	8.2	7.0	8.9
2	23456789B	Ana García	8.0	7.1	7.9	6.3	7.4
3	34567890C	Carlos Martínez	6.5	8.5	8.0	7.8	9.1
5	45678901D	Luisa Sánchez	9.2	6.3	9.0	6.9	8.6
8	56789012E	María Rodríguez	5.7	5.8	6.5	8.1	7.0

Acceso mediante loc

In [12]:
```
print("Acceso con loc:")
df_alumnos.loc[1, 'Nombre y apellidos']
```

```
Acceso con loc:
```

Out[12]:
```
'Juan Pérez'
```

Acceso mediante iloc

In [13]:
```
print("Acceso con iloc:")
df_alumnos.iloc[1, 1]
```

```
Acceso con iloc:
```

Out[13]:
```
'Ana García'
```

Por último, también podemos filtar las filas, seleccionando aquellas que cumplan una o varias condiciones. Por ejemplo:

In [14]:
```
# Filtramos las filas con "Nota matematicas" < 5
filas_filtradas = df_alumnos[df_alumnos['Nota matematicas'] < 5]

filas_filtradas
```

Out[14]:

	DNI	Nombre y apellidos	Nota lengua	Nota matematicas	Nota ingles	Nota fisica	Nota quimica
10	67890123F	Pedro López	8.9	4.2	7.1	7.7	8.4
12	78901234G	Laura Fernández	7.2	2.7	8.3	6.5	7.6

Ejercicio 5

1. Cambia el índice del DataFrame "df_alumnos" por la columna DNI.
2. Muestra la nota en la asignatura lengua del alumno con DNI "90123456I".

3. Muestra la nota en la asignatura lengua del alumno de la fila con la posición 8.

4.2.Estructuras de control

4.2.1.If

La estructura if permite controlar el flujo de ejecución del programa mediante condiciones. Un if se usa para tomar decisiones, es decir, ejecutar cierto bloque de código si una condición especificada se evalúa como verdadera (True) y, opcionalmente, ejecutar otro bloque de código si la condición es falsa (False).

In [15]:
```
edad = 18

if edad >= 18:
    print("Eres mayor de edad.")
else:
    print("Eres menor de edad.")

```

```
Eres mayor de edad.
```

4.2.3.For

Un bucle for se utiliza para iterar sobre una secuencia de elementos (como una lista o un array) o repetir un bloque de código un número específico de veces. Los bucles for se suelen utilizar para recorridos, por lo tanto, se iterará hasta recorrer todos los elementos de la secuencia o repetir un bloque de código un número de veces especificado.

A continuación, tienes un ejemplo para repetir un bloque de código un número específico de veces:

In [16]:
```
for i in range(0, 10):
    print("Hola mundo")
```

```
Hola mundo
Hola mundo
Hola mundo
Hola mundo
Hola mundo
Hola mundo
Hola mundo
Hola mundo
Hola mundo
Hola mundo
```

En la siguiente celda tienes un bucle for para iterar sobre una lista de elementos.

In [17]:

```python
# Creamos un array de 8 números
numeros = [2, 4, 6, 8, 10, 12, 14, 16]

# Inicializamos una variable para la suma
suma = 0

# Utilizamos un bucle for para sumar los números en el array
for numero in numeros:
    suma += numero

# Mostramos la suma
print("La suma de los números en el array es:", suma)

```

```
La suma de los números en el array es: 72
```

4.2.2.While

Con el bucle while podemos ejecutar un conjunto de sentencias mientras una condición sea cierta. Los bucles while suelen utilizarse para búsquedas, esto quiere decir que en el momento en el que se cumpla una determinada condición se debe terminar la ejecución del bucle, aunque queden elementos por recorrer.

In [30]:

```python
# Lista de números
numeros = [1, 3, 5, 7, 9, 11]

# Número que queremos comprobar si está en la lista
numero_buscar = 5

# Inicializamos una variable de índice
indice = 0

# Inicializamos una variable para indicar si se encontró el número
encontrado = False

# Usamos un bucle while para recorrer la lista
while ( (indice < len(numeros)) and (not encontrado) ):

    if numeros[indice] == numero_buscar:
        encontrado = True
    else:
      indice += 1

# Comprobamos si hemos encontrado el número
if encontrado:
    print("El número", numero_buscar, "se encuentra en la lista.")
else:
    print("El número", numero_buscar, "no se encuentra en la lista.")

```

```
El número 5 se encuentra en la lista.
```

Ejercicio 6

1. Crea un programa que introduzca números enteros aleatorios del 1 al 100 en una lista hasta que la suma de los números sea mayor o igual que 1000.
2. Modifica el programa anterior para que si se genera el número 33 aparezca un mensaje indicando de que se ha añadido el número de la suerte a la lista.

4.3.Funciones

Una función es un segmento de código separado del bloque principal y que puede ser invocado en cualquier momento desde este o desde otra función. Cuando se llama a una función, el código que se estaba ejecutando se detiene y se dirige a ejecutar el código de la función.

In [19]:
```
def calcular_cuadrado(numero):
    cuadrado = numero ** 2
    return cuadrado

# Ejemplo de uso:
numero = 5
resultado = calcular_cuadrado(numero)
print("El cuadrado de", numero, "es", resultado)

```

```
El cuadrado de 5 es 25
```

Ejercicio 7

Escribe una función que encapsule el código del ejercicio 6 teniendo como argumentos el valor mínimo que debe sumar la lista y el número mágico.

Anexo: Funcionalidades adicionales de los DataFrames

Group by

La función groupby() de Pandas se utiliza para agrupar las filas de un DataFrame según los valores de una o más columnas. Esta función es especialmente útil cuando necesitamos realizar operaciones estadísticas o de agregación en subconjuntos de datos que comparten un valor común en una o varias columnas.

In [20]:

```
import pandas as pd

# Creamos un DataFrame de ejemplo
data = {'Tienda': ['Tienda A', 'Tienda B', 'Tienda C', 'Tienda D', 'Tienda
        'Tamaño': ["Pequeña", "Grande", "Mediana", "Grande", "Pequeña"],
        'Facturación Anual': [1000000, 1500000, 1200000, 1800000, 900000]}

df_tiendas = pd.DataFrame(data)
df_tiendas
```

Out[20]:

	Tienda	Tamaño	Facturación Anual
0	Tienda A	Pequeña	1000000
1	Tienda B	Grande	1500000
2	Tienda C	Mediana	1200000
3	Tienda D	Grande	1800000
4	Tienda E	Pequeña	900000

In [21]:

```
# Usamos groupby para agrupar por la columna 'tamaño' y calcular la factur
resultados = df_tiendas.groupby('Tamaño')['Facturación Anual'].mean()
print(resultados)
```

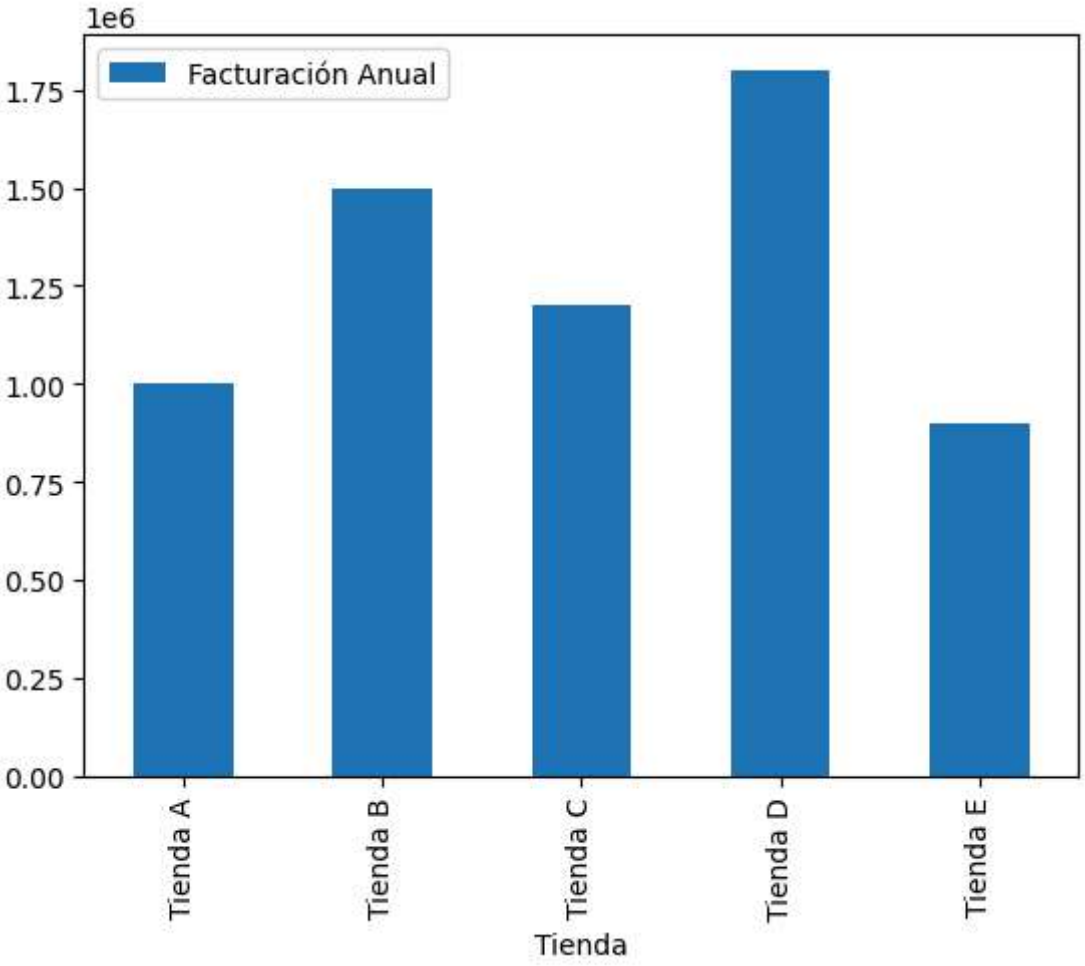
```
Tamaño
Grande     1650000.0
Mediana    1200000.0
Pequeña     950000.0
Name: Facturación Anual, dtype: float64
```

Gráficos

Pandas no incluye directamente funciones para crear gráficos. Sin embargo, Pandas se integra estrechamente con la biblioteca de visualización de datos matplotlib. A continuación, tienes algunos ejemplos.

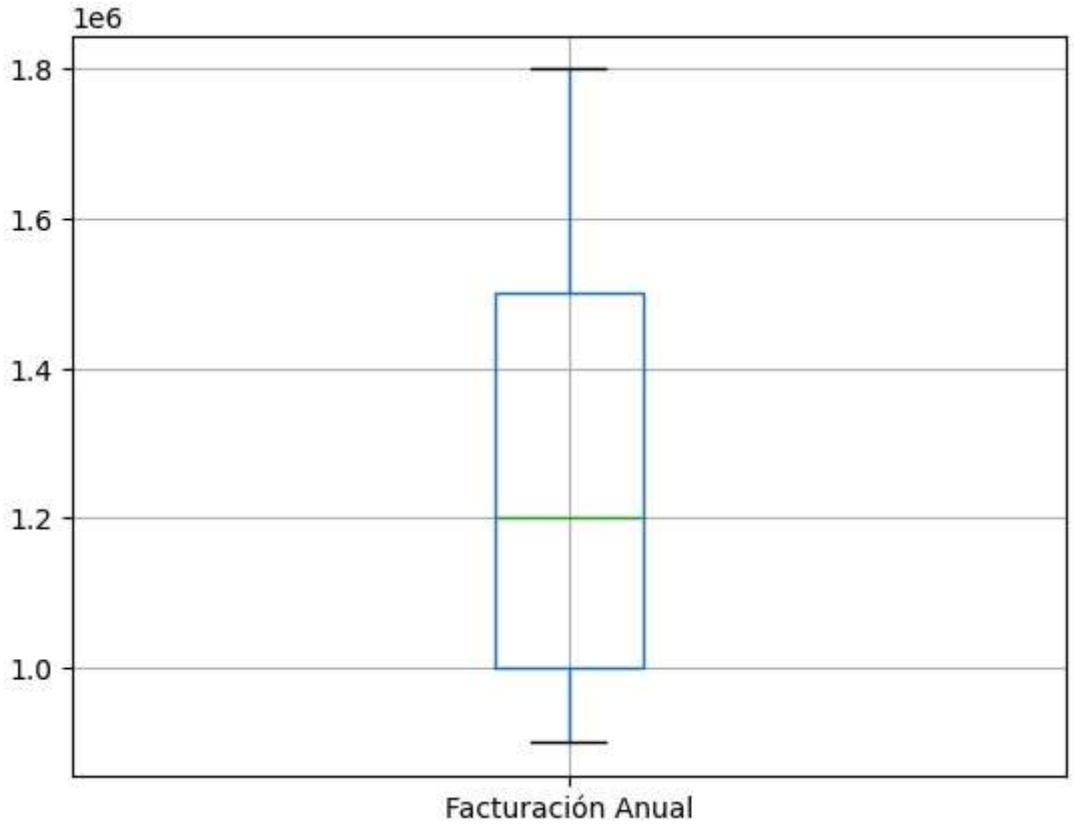
In [22]:
```
1 df_tiendas.plot("Tienda", 'Facturación Anual', kind='bar')
```

Out[22]: <Axes: xlabel='Tienda'>



In [23]:
```
1 df_tiendas.boxplot("Facturación Anual")
```

Out[23]: <Axes: >



Descripciones generales

Puedes obtener descripciones generales de un DataFrame mediante las funciones *info* y *describe*.

In [24]:
```
1 df_tiendas.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column             Non-Null Count  Dtype 
---  ------             --------------  ----- 
 0   Tienda             5 non-null      object
 1   Tamaño             5 non-null      object
 2   Facturación Anual  5 non-null      int64 
dtypes: int64(1), object(2)
memory usage: 252.0+ bytes
```

In [25]:
```
1 df_tiendas.describe()
```

Out[25]:

	Facturación Anual
count	5.000000e+00
mean	1.280000e+06
std	3.701351e+05
min	9.000000e+05
25%	1.000000e+06
50%	1.200000e+06
75%	1.500000e+06
max	1.800000e+06

Leer y guardar DataFrames

En este notebook hemos creado DataFrames muy pequeños para los distintos ejemplos. Sin embargo, esto no será lo habitual en las prácticas de la asignatura. Por ello tendremos que cargar conjuntos de datos mediante *pd.read_csv("ruta_al_fichero_y_nombre.csv")*.

También podemos guardar un DataFrame, por ejemplo, porque hayamos hecho cambios en él como eliminar los valores faltantes (NA). Para guardar un DataFrame podemos hacer uso de la función *to_csv("ruta_al_fichero_y_nombre.csv")* de Pandas.

In [26]:
```
1 # Cargamos un DataFrame de ejemplo incluido en Google Colab
2 df_ejemplo = pd.read_csv("/content/sample_data/california_housing_test.csv
```

In [27]:
```
1 df_ejemplo
```

Out[27]:

	longitude	latitude	housing_median_age	total_rooms	total_bedrooms	population	household
0	-122.05	37.37	27.0	3885.0	661.0	1537.0	60
1	-118.30	34.26	43.0	1510.0	310.0	809.0	27
2	-117.81	33.78	27.0	3589.0	507.0	1484.0	49
3	-118.36	33.82	28.0	67.0	15.0	49.0	1
4	-119.67	36.33	19.0	1241.0	244.0	850.0	23
...	...	...	...	...	...	...	
2995	-119.86	34.42	23.0	1450.0	642.0	1258.0	60
2996	-118.14	34.06	27.0	5257.0	1082.0	3496.0	103
2997	-119.70	36.30	10.0	956.0	201.0	693.0	22
2998	-117.12	34.10	40.0	96.0	14.0	46.0	1
2999	-119.63	34.42	42.0	1765.0	263.0	753.0	26

3000 rows × 9 columns

In [28]:
```
1 from google.colab import drive
2 drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call
drive.mount("/content/drive", force_remount=True).
```

In [29]:
```
1 # Guardamos el DataFrame en nuestro Google Drive
2 df_ejemplo.to_csv("/content/drive/MyDrive/df_ejemplo_p1.csv")
3
```